RESEARCH & PROTOTYPE REVIEW

Machine Learning & Deep Learning Algorithm Audit

Source-code audit of actual scheduling logic, AI integration attempts, predictive-maintenance screen and future ML path.

SIH 2026 | PS SIH26250 | Alchemy Architects

SOURCE-CODE REVIEW + PUBLIC RESEARCH | 20 OCTOBER 2026

ALCHEMY ARCHITECTS

Executive conclusion

No verified machine-learning or deep-learning training/inference pipeline was found in the supplied source paths reviewed. The main computational algorithm is a deterministic scheduling heuristic, supported by a separately coded rule-based validator. The UI labels a section “Multi-Variate Degradation Regressor” and displays RUL metrics, but source review shows those values are hard-coded in component/API/seed code. Treat them as illustrative display claims, not measured results.

1. What algorithms are actually present?

Algorithm / component	What the code does	Classification	Caveat
Priority-ranked deterministic optimizer	Filters non-AVAILABLE resources, orders tasks by priority then deadline, searches non-overlapping slots, scores candidates using delay, deadline penalty, load balance and assignment churn.	Heuristic optimisation / scheduling	Not a trained ML model; no OR-Tools dependency observed in the exported package.
Greedy first-fit baseline	Processes tasks in input order and assigns the earliest feasible available resource based on capability and time windows.	Greedy heuristic baseline	Baseline objective formula is not perfectly aligned with the full composite score; benchmark interpretation needs review.
Independent constraint validator	Checks resource existence/readiness, availability window, maintenance blackouts, no overlapping assignments, capability match, dependencies and freshness warning state.	Deterministic rule-based verification	Inspected code does not visibly test every claimed property; see gaps below.
Data-quality rules	Flags seeded stale/conflicting/unknown records and displays quality issue categories.	Rules and seeded examples	Verify that displayed issues are derived from data rather than fixed counts/messages.
VAYU NEXA AI	Server contains optional Gemini request logic with an offline rule-based fallback.	Generative AI integration attempted + deterministic fallback	Live model identifiers, API key, returned grounding and successful calls have not been independently verified.
C-MAPSS RUL screen	Displays five seeded sensor/remaining-useful-life rows and fixed baseline/candidate metrics/feature weights.	Static demonstration data	No code for fitting/training a regression model or calculating those displayed metrics was found.

2. How the deterministic scheduler scores options

The inspected `src/solver/optimizer.ts` creates feasible candidate slots for each priority-sorted task and chooses the candidate with the lowest composite score. The code sets default weights of delay 1.5, unassigned penalty 10, churn 2, and utilisation 1. Candidate score includes delay x delayWeight, a 500 deadline-miss penalty, load-balance penalty of 10 per previously scheduled task on that resource, and churn penalty of churnWeight x 50. The overall objective adds unassignedCount x unassignedWeight x 100, delay and churn terms, then subtracts average utilisation x utilisationWeight.

These formulas describe a deterministic heuristic. They are not the objective of an external CP-SAT solver, and a status labelled OPTIMAL when no tasks are unassigned does not mathematically prove global optimality.

3. Independent validator: strengths and gaps

- Strengths present in code: rejects assignment to a resource that is not AVAILABLE; verifies resource availability window; checks maintenance collision, overlapping resource allocations, required capabilities, prerequisite order and warning-state provenance.

- Gap to verify/fix: the inspected validator does not visibly check that each assignment references a known task, that all tasks are accounted for exactly once, that assignment duration equals task duration, that start time meets earliestStart, or that the assignment ends by latestDeadline.
- Gap to verify/fix: task sort is priority/deadline based although a source comment describes topological ordering. A prerequisite with lower priority than its dependent can be processed later and lead to an avoidable unassigned task.
- Potential benchmark gap: baseline score and candidate objective use different weights/formulas, so any displayed “improvement percentage” may not be a like-for-like comparison.
- The source constructs a timestamp-derived field named inputSnapshotHash; this is not a cryptographic hash of the serialized input records. A real reproducibility hash should be computed from canonical input JSON.

4. ML/DL claims displayed in the UI: evidence check

Displayed item	Where observed	Audit conclusion
Baseline RMSE 34.2 and MAE 28.5	`src/components/views/PredictiveMaintenanceLab.tsx`, `ModelEvaluation.tsx`, `/api/nasa-cmapss` response	Hard-coded values in the inspected files. Experiment and evaluation evidence not included.
Candidate RMSE 14.8 and MAE 10.6	Same UI/API paths	Hard-coded values; model architecture named but no reproducible model implementation found.
95% interval +/- 12.4 cycles	UI/API display value	No code that computes or calibrates this confidence interval was found.
Sensor feature weights (T50 28%, P30 22%, etc.)	Static `sensorWeights` array in ModelEvaluation.tsx	Presentation weights, not trained/importances computed from a fit model.
Seeded RUL predicted values	`NASA_CMAPSS_RECORDS` in `src/data/seedData.ts`	Prefilled example row values; not produced by a prediction function at runtime in the inspected view.

5. Recommended ML roadmap (future work, not present capability)

Stage	Required implementation/evidence	Output
Dataset lock	Download exact NASA C-MAPSS FD001 archive; record citation, checksum, licence/terms and README.	Versioned immutable dataset.
Preprocess	Parse engine trajectories, select sensor channels, handle operating conditions, scale using training-only statistics, define RUL target and optional cap.	Versioned preprocessing pipeline.
Split correctly	Split by engine/unit so the same engine does not leak between train/validation/test. Record seed.	Reproducible split manifest.
Baseline	Implement a documented naive/statistical baseline and calculate metrics from held-out predictions.	RMSE, MAE and residual plots.
Candidate model	Choose a simple regression model first, such as Random Forest or gradient-boosted trees; keep deep learning optional until data volume/need justify it.	Serialized model and training config.
Evaluation	Compute metrics from actual predictions, compare matched held-out set, report uncertainty method and failure examples.	Versioned evaluation JSON and test script.
Integration	Serve predictions through a typed endpoint with dataset/model version, timestamp, uncertainty and non-operational label.	Traceable RUL output with explicit limits.

6. Full current feature inventory (source bundle, not live verification)

The source tree includes modules or navigation items for: Command Centre and 3D constellation; readiness registry; workforce/equipment/inventory views; data-quality studio; planning sandbox; dynamic replanning and disruption timeline; public weather connector view; predictive maintenance and model evaluation screens; VAYU NEXA AI; evidence ledger/audit events; alerts/review; security/role-admin UI; 35-item requirements coverage; accessibility/help; and aerospace/bioscience or aircraft-library visual screens. Code presence is not proof each module works end-to-end. Several original modules are operationally themed and should be excluded from a public safe-demo scope unless they are reduced to generic, synthetic, non-operational examples.

PPT-safe wording: “The current source implements deterministic scheduling and rule-based validation. Predictive maintenance is a UI/seed-data experiment at this stage; a trained and reproducibly evaluated ML/DL model remains future work.”

References and source notes

[1] NASA Prognostics Center of Excellence Dataset Repository
https://www.nasa.gov/intelligent-systems-division/discovery-and-systems-health/pcoe/pcoe-data-set-repository/
Official NASA repository lists turbofan engine degradation simulation data and the C-MAPSS benchmark. It is not military telemetry.

[2] NASA C-MAPSS Dataset Citation / Description
https://c3.ndc.nasa.gov/dashlink/resources/139/
NASA-origin dataset description explains run-to-failure engine degradation simulation; the user app’s five hard-coded sample rows are not, by themselves, proof of ingestion of the dataset.

[3] Google OR-Tools CP-SAT Solver
https://developers.google.com/optimization/cp/cp_solver
Useful comparison for a future solver if exact constraint optimisation is needed; the inspected source is a custom deterministic heuristic.